گزارش پروژهٔ درس هوش مصنوعی

حل مسئلهٔ جعبه‌های شن با الگوریتم‌های جستجو

مدل پایه: بدون آب و بتن

گزارش کار با تغییرات درخواستی استاد در هفته دوم

دانشجو: پرتو روشنی

استاد: دکتر کتانفروش

سال تحصیلی: ۱۴۰۶--۱۴۰۵
نیمسال اول

گزارش پیاده‌سازی، اعتبارسنجی و ارزیابی تجربی

فهرست مطالب

۱ مقدمه و محدوده تغییرات هفته دوم ۲

۱.۱ موضوع گزارش ۲

۲.۱ نمای کلی تغییرات نسبت به گزارش اول ۲

۳.۱ هدف‌های ارزیابی ۳

۲ مدل‌سازی مسئله پس از اصلاح قواعد ۴

۱.۲ خانه‌ها، وضعیت اولیه و شرط هدف ۴

۲.۲ قواعد نهایی دریچه‌های عمودی و افقی ۴

۳.۲ تابع هزینه اصلاح‌شده ۵

۴.۲ مشکلی که در میانه اصلاحات پیدا شد ۵

۳ شرح مرحله‌به‌مرحله کارهای هفته دوم ۶

۱.۳ هماهنگ‌سازی قرارداد ورودی و نمایش ۶

۲.۳ تثبیت وضعیت دریچه‌های باز و انتقال خودکار ۶

۳.۳ رفع ناسازگاری Greedy و مثال استاد ۶

۴.۳ به‌روزرسانی اجرای یکپارچه و خروجی‌ها ۷

۵.۳ ساخت تابع تخمین به روش Relaxation ۷

۶.۳ آموزش Heuristic با یادگیری ماشین ۷

۷.۳ کاهش سربار پیش‌بینی و اندازه‌گیری حافظه ۷

۸.۳ رفع خطاهای انتقال نسخه‌ها و اسکریپت‌های اجرایی ۷

۹.۳ افزودن الگوریتم Random بدون تکرار وضعیت ۸

۴ الگوریتم‌های جست‌وجو و معیار مبنا ۹

۱.۴ Greedy استاد: تخلیه مستقل ستون‌ها ۹

۲.۴ :Random جست‌وجوی عمقی تصادفیِ بدون بازدید مجدد ۹

۳.۴ دیگر الگوریتم‌ها ۹

۵ بازاجرای مثال استاد ۱۱

۱.۵ شبکه اولیه و مسیر Greedy ۱۱

۲.۵ مقایسه با جواب کم‌هزینه ۱۲

۳.۵ جدول هر هشت الگوریتم در اجرای یکپارچه ۱۲
۴.۵ چرا نتیجه Random در دو دستور متفاوت بود؟ ۱۳

۶ طراحی تابع تخمین هزینه به روش Relaxation ۱۴
۱.۶ تابع Basic و ایده پیشنهادشده در کلاس ۱۴
۲.۶ فرمول پیشنهادی و پیاده‌سازی آن ۱۴
۳.۶ استدلال برای Admissibility ۱۵
۴.۶ اعتبارسنجی محاسباتی و اثر بر A* ۱۵

۷ یادگیری ماشین برای تخمین هزینه باقیمانده ۱۶
۱.۷ تولید داده برچسب‌دار ۱۶
۲.۷ تقسیم داده و جلوگیری از هم‌پوشانی آرایش‌ها ۱۶
۳.۷ معیارهای پیش‌بینی ۱۷
۴.۷ دو روش استفاده از ML در A* ۱۷

۸ نتایج جامع، نمودارها و مقایسه روش‌ها ۱۸
۱.۸ تحلیل دقیق تمام شبکه‌های اولیه ۳×۲ . . . . . ۱۸
۲.۸ مقایسه هشت الگوریتم روی پنج ورودی نمونه ۵×۲ . . . . . ۲۰
۳.۸ آزمایش ۳۰ Seed برای Random روی مثال استاد ۲۱
۴.۸ ارزیابی پنج روش Heuristic روی ۶۰ ورودی ۲۳
۵.۸ برآورد ضریب انشعاب مؤثر b^* ۲۴
۶.۸ آزمایش حافظه با سقف ۶۰ ثانیه ۲۶

۹ اعتبارسنجی، محدودیت‌ها و بازتولید نتایج ۲۸
۱.۹ آزمون‌های صحت پس از تغییرات ۲۸
۲.۹ محدودیت‌های تجربی ۲۸
۳.۹ فایل‌های اصلی و نقش آن‌ها ۲۹
۴.۹ دستورهای اصلی بازتولید در PowerShell ۲۹

۱۰ نتیجه‌گیری هفته دوم ۳۱

آ منابع داده و پیوست‌های قابل بررسی ۳۲

فصل ۱

مقدمه و محدوده تغییرات هفته دوم

۱۰۱ موضوع گزارش

این گزارش ادامه گزارش اولیه پروژه درس هوش مصنوعی درباره تخلیه جعبه‌های شن با استفاده از الگوریتم‌های جست‌وجو است. ساختار کلی و اطلاعات جلد از گزارش پیشین حفظ شده، اما متن حاضر بر **تمام اصلاحات و آزمایش‌های انجام‌شده در هفته دوم** تمرکز دارد. به درخواست استاد، قواعد حرکت شن و تعریف روش مبنای حریصانه بازبینی شد؛ نمایش ورودی اصلاح شد؛ الگوریتم تصادفی بدون بازدید مجدد وضعیت‌ها اضافه شد؛ دو راه برای طراحی تابع تخمین هزینه، یعنی ساده‌سازی مسئله و یادگیری ماشین، توسعه یافت و نتایج آن‌ها جداگانه بررسی شد.

در تمام گزارش، مسئله فقط **مدل پایه شن، بدون آب و بتن** است. گزارش قبلی حاوی برخی آزمایش‌های متعلق به نسخه قدیمی مدل بود. نتایج عددی این نسخه‌ها را با مدل اصلاح‌شده ادغام نمی‌کنیم؛ مگر در جایی که هدف، توضیح روند تغییرات و علت یک خطا باشد.

۲۰۱ نمای کلی تغییرات نسبت به گزارش اول

موضوع	وضعیت یا ابهام اولیه	تغییر تثبیت‌شده در هفته دوم
قرارداد خانه‌ها	وجود نمایش برعکسِ مقدار پر و خالی در بعضی بخش‌ها	مقدار ۱ پر، ۰٫۵ نیمه‌پر و ۰ خالی؛ هماهنگی خط فرمان، خروجی و انیمیشن
دریچه‌های عمودی	بسته شدن زودهنگام بعد از اولین انتقال، و نتیجه غیرمرجع برای Greedy	جریان پیوسته، انتظار دریچه مسدود و تخلیه زنجیره‌ای بدون عمل اضافی

دریچه‌های افقی	نمایش و وضعیت ماندگار ناقص	انتقال حداکثر نیم‌واحد، حفظ دریچه مسدود و انتقال خودکار بعدی
Greedy	احتمال اشتباه گرفتن با جست‌وجوی حریصانه آگاهانه	تعریف صریح تخلیه ستون‌به‌ستون، فقط با دریچه عمودی
Random	در مقایسه الگوریتم‌ها وجود نداشت	جست‌وجوی عمقی تصادفی با مجموعه وضعیت‌های دیده‌شده و Seed ثابت
Heuristic	تابع پایه با تخمین پایین	تابع Relaxation مبتنی بر مرزهای ردیف و دریچه باز
یادگیری ماشین	وجود نداشت	آموزش Random Forest با برچسب هزینه دقیق و دو شیوه استفاده در A*
اعتبارسنجی	نمونه‌ها و آزمون‌های نسخه قدیمی	تست‌های بازگشتی، تحلیل دقیق × ۳ ۲، آزمایش چند Seed و اندازه‌گیری حافظه

۳.۱ هدف‌های ارزیابی

هدف، کمینه کردن مجموع هزینه وزن‌دار اعمال برای خالی کردن کامل شبکه است. افزون بر هزینه، تعداد اعمال، تعداد وضعیت‌های گسترش‌یافته، زمان جست‌وجو، حافظه و ضریب انشعاب مؤثر b^* ثبت می‌شود. برای روش تصادفی، وابستگی نتیجه به Seed بررسی می‌شود. برای A*، دو موضوع جداست: اعتبار ریاضی تابع تخمین و سودمندی عملی آن در کاهش تعداد گسترش‌ها.

فصل ۲

مدل‌سازی مسئله پس از اصلاح قواعد

۱.۲ خانه‌ها، وضعیت اولیه و شرط هدف

شبکه‌ای با R ردیف و C ستون داریم. هر خانه حداکثر یک واحد شن نگه می‌دارد و مقدار آن در ورودی یکی از ۱ ,۰٫۵ ,۰ است. همه دریچه‌ها **در وضعیت اولیه بسته‌اند.** مقدار ۱ به معنای خانه پر و ۰ به معنای خانه خالی است؛ بنابراین شبکه کاملاً خالی هدف مسئله است:

$$\mathrm{Goal}(s) \iff \forall i \; q_i = ۰.$$

در پیاده‌سازی، مقدار شن هر خانه به صورت یک رقم صحیح $u_i \in \{۰, ۱, ۲\}$ نگه داشته می‌شود که برابر دو برابر مقدار فیزیکی شن است. ماسک بیت دریچه‌های باز نیز در وضعیت گنجانده می‌شود. پس دو آرایش با شن یکسان اما دریچه‌های متفاوت، وضعیت‌های یکسانی نیستند.

$$S = (u_۰, \ldots, u_{RC-۱}, M), \qquad \mathrm{encode}(S) = \sum_{i=۰}^{RC-۱} u_i ۳^i + ۳^{RC} M.$$

این نمایش برای جلوگیری از تکرار وضعیت‌ها در Random، BFS و سایر جست‌وجوهای گرافی اهمیت دارد.

۲.۲ قواعد نهایی دریچه‌های عمودی و افقی

۱. **دریچه عمودی:** باز کردن آن یک عمل هزینه‌دار است. اگر مقصد ظرفیت داشته باشد، شن به پایین منتقل می‌شود. اگر مقصد پر باشد، دریچه باز می‌ماند؛ سپس هر وقت مقصد ظرفیت پیدا کند، جریان خودکار ادامه می‌یابد.

۲. **جریان پیوسته:** دریچه داخلی تا وقتی پشت آن شن باقی مانده و انتقال در همان زنجیره امکان ادامه یا انتظار دارد، می‌تواند باز بماند. بسته شدن آن به پایان جریان وابسته است، نه صرفاً نخستین عبور شن.

۳. **خروج از پایین:** با باز شدن دریچه خروجی، شن خانه پایین خارج می‌شود و اگر شن بالاتر از دریچه‌های باز به آن برسد، در جریان پیوسته همان عمل تخلیه می‌شود. برای این انتقال‌های ناشی از عمل اصلی، هزینه باز کردن دریچه تازه محاسبه نمی‌شود.

۴. **دریچه افقی:** در هر بار باز کردن، حداکثر نیم‌واحد شن عبور می‌کند. در مدل جاری، باز کردن فعال دریچه افقی از مبدأ پر انجام می‌شود. دریچه مسدود می‌تواند باز بماند و با ایجاد ظرفیت در مقصد، انتقال خودکار نیم‌واحدی انجام شود.

۵. **استقلال اعمال و جریان خودکار:** هزینه مربوط به عملِ انتخاب و باز کردن دریچه است؛ حرکت‌های خودکار در همان زنجیره، اعمال تازه در مسیر جست‌وجو محسوب نمی‌شوند.

۳.۲ تابع هزینه اصلاح‌شده

اگر q_{before} و q_{after} مقدار فیزیکی شن در خانه مبدأ دریچه عمودی انتخاب‌شده پیش و پس از عمل باشند، هزینه آن چنین است:

$$C_{\text{down}}(a) = ۱ + ۴\big(q_{\text{before}} + q_{\text{after}}\big), \qquad C_{\text{side}}(a) = ۱.$$

برای نمونه، هزینه تخلیه یک خانه پر که در پایان عمل خالی شده، ۵ است؛ اگر با باز کردن یک دریچه، مبدأ پر باقی بماند، هزینه آن ۹ می‌شود. هزینه عملِ خروج نباید دوباره برای تک‌تک خانه‌هایی که در ادامه جریان از شبکه خارج می‌شوند شمرده شود. همین تفاوت در تفسیر تابع هزینه، در بازتولید مثال استاد تعیین‌کننده بود.

۴.۲ مشکلی که در میانه اصلاحات پیدا شد

در یک نسخه میانی، دریچه پس از نخستین عبور شن بسته می‌شد؛ درنتیجه برای بعضی خانه‌ها باز کردن دریچه خروجی تکرار می‌شد و هزینه Greedy مثال استاد به ۶۶ رسیده بود. بررسی دستی نشان داد این نتیجه با شرط جریان پیوسته تطابق ندارد. پس منطق انتقال و پایان جریان در `model.py` اصلاح شد و آزمون بازگشتی برای هزینه ۴۷ افزوده شد. این عدد در کد ثابت گذاری نشده است؛ از اجرای خود قوانین به دست می‌آید.

فصل ۳

شرح مرحله‌به‌مرحله کارهای هفته دوم

۱.۳ هماهنگ‌سازی قرارداد ورودی و نمایش

نخست مشخص شد قرارداد موردنظر مسئله = ۱پر، = ۰٫۵نیمه‌پر و = ۰خالی است. در برخی نسخه‌های اولیه، ۰ و ۱ معنای معکوس داشتند. نگاشت نمایش در مدل و گزینه پیش‌فرض CLI اصلاح شد. برچسب اعداد در HTML و GIF نیز یکسان شد؛ یک خطای جداگانه باعث می‌شد GIF خانه خالی را با ۱ نشان دهد که تصحیح شد. نمایش داخلی ۰٫۱٫۲ برای اشغال خانه باقی ماند، زیرا برای محاسبه انتقال در مقیاس نیم‌واحدی مناسب است.

۲.۳ تثبیت وضعیت دریچه‌های باز و انتقال خودکار

پس از روشن شدن تفسیر استاد، وضعیت دریچه‌های بازِ عمودی و افقی داخل کدگذاری وضعیت ذخیره شد. مسیرهای متوالی جریان، دریچه‌های منتظر ظرفیت و پایان جریان در تابع انتقال یکپارچه شدند. اکنون ثبت رویدادهای انتقال به تفکیک direct و automatic انجام می‌شود و رسم وضعیت دریچه‌های باز در انیمیشن ممکن است.

۳.۳ رفع ناسازگاری Greedy و مثال استاد

روش مبنای استاد یک روش از پیش مشخص برای تخلیه مستقل ستون‌هاست؛ Greedy Best-First Search نیست. پس از بازتعریف جریان پیوسته، هزینه مثال چهارردیفی در هفت عمل به ۴۷ بازگشت. همچنین هزینه بهینه UCS و A* برابر ۳۴ باقی ماند. شرط قدیمی Greedy==47 به عنوان یک مقدار سخت کدشده جای خود را به آزمون صحت مدل داد.

۴.۳ به‌روزرسانی اجرای یکپارچه و خروجی‌ها

فایل run_project.py برای اجرای آزمایش‌های نسخه اصلاح‌شده به‌روز شد؛ تحلیل کامل ۲ × ۵ که متعلق به نسخه قدیم بود دیگر بدون بررسی نسخه مدل بازاستفاده نمی‌شود. نسخه جدید داده‌ها را در پوشه مجزا ثبت می‌کند. برای تحلیل دقیق و نمودارهای نسبت هزینه، شبکه ۲ × ۳ انتخاب شد؛ این انتخاب به دلیل امکان شمارش تمام وضعیت‌های پایدار و اجرای Reverse Dijkstra بود، نه به معنای حل کامل فضای حالت بزرگ‌تر.

۵.۳ ساخت تابع تخمین به روش Relaxation

بعد از بررسی تابع پایه، یک تابع پیشنهادی کلاس برحسب شمار خانه‌های پر و نیمه‌پر مطرح شد. به دلیل امکان تخلیه چند خانه در یک جریان پیوسته، این تابع در برخی وضعیت‌های میانی بیش‌برآورد می‌کند. سپس تخمین مبتنی بر مرزهای ردیفی طراحی شد که دریچه‌های باز و شن بالای هر مرز را لحاظ می‌کند؛ این تابع با Reverse Dijkstra اعتبارسنجی شد.

۶.۳ آموزش Heuristic با یادگیری ماشین

برای شبکه ۲ × ۳، برچسب هزینه دقیق باقی‌مانده برای همه وضعیت‌های پایدار تولید شد. الگوریتم RandomForestRegressor با ویژگی‌های شن و ماسک دریچه‌ها آموزش دید. مدل در سه تقسیم مستقل آموزش/آزمون با Seedهای ۴۲, ۴۳, ۴۴ آزمایش شد؛ دو شیوه ML Direct و ML Safe Tie نیز در جست‌وجو اضافه شدند.

۷.۳ کاهش سربار پیش‌بینی و اندازه‌گیری حافظه

ابتدا فراخوانی مکرر مدل ML زمان زیادی مصرف می‌کرد. در نسخه بعد، پیش‌بینی‌های تکراری در حافظه همان جست‌وجو نگه داشته شد و وضعیت‌های جدید به صورت دسته‌ای پیش‌بینی شدند. یک آزمایش مستقل با --measure-memory انجام شد تا حافظه تخصیص‌یافته Python اندازه‌گیری شود. چون tracemalloc خود زمان اجرا را افزایش می‌دهد، نتایج زمانی بدون پایش حافظه جدا نگه داشته شدند. افزایش سقف زمانی از ۲۰ به ۶۰ ثانیه در اجرای پروفایل‌شده، همه ۶۰ آزمایش ML Safe Tie را به پایان رساند.

۸.۳ رفع خطاهای انتقال نسخه‌ها و اسکریپت‌های اجرایی

در طول اعمال فایل‌های اصلاحی، چند خطای اجرایی نیز تشخیص داده و رفع شد. نخست فایل apply_input_convention.py در پوشه اشتباه قرار گرفته بود؛ با نهایی شدن قرارداد در خود model.py و cli.py، دیگر اجرای این اسکریپت جداگانه لازم نبود. در مرحله بعد، نسخه ناقصی از

sandgrid/search.py درپروژه قرار گرفت و وارد کردن SearchResult و greedy شکست خورد؛ فایل جست‌وجوی صحیح جایگزین شد و مجموعه تست دوباره پاس شد. در مرحله ML نیز یک نسخه نادرست از final_heuristic_study.py گزینه‌های seeds-- و count-- را نمی‌شناخت؛ با جایگزینی نسخه درست و بررسی help--، آزمایش چند Seed انجام شد. این موارد خطاهای یکپارچه‌سازی فایل‌ها بودند، نه تغییر جدید در قانون فیزیکی مسئله.

۹.۳ افزودن الگوریتم Random بدون تکرار وضعیت

در آخر، به درخواست درس یک جست‌وجوی تصادفی اضافه شد. این روش ترتیب جانشین‌های قانونی را با Seed مخلوط می‌کند، اما با مجموعه visited اجازه کشف دوباره وضعیت کامل را نمی‌دهد. امکان عقب‌گرد در بن‌بست، ذخیره مسیر والدها، اجرای تکی، ساخت انیمیشن و حضور در بنچمارک هشت‌الگوریتمی فراهم شد. برای بررسی تغییرپذیری آن، سی اجرای مستقل با Seedهای ۱ تا ۳۰ روی مثال استاد ثبت شد.

فصل ۴

الگوریتم‌های جست‌وجو و معیار مبنا

۱۰۴ Greedy استاد: تخلیه مستقل ستون‌ها

در روش Greedy استاد، هیچ دریچه افقی باز نمی‌شود. از ستون اول شروع می‌کنیم، بالاترین خانه دارای شن را پیدا می‌کنیم، دریچه زیر آن را باز می‌کنیم و از بالا به پایین پیش می‌رویم. اگر دریچه مسدود شود، باز می‌ماند و با ایجاد ظرفیت در مقصد، انتقال می‌تواند خودکار ادامه یابد. پیمایش تا تخلیه کامل ستون تکرار می‌شود؛ سپس ستون بعدی بررسی می‌شود. این روش صرفاً یک **Baseline مشخص** است و تضمین کمینه کردن هزینه ندارد.

۲۰۴ :Random جست‌وجوی عمقی تصادفیِ بدون بازدید مجدد

الگوریتم Random پیاده‌سازی‌شده نوعی Randomized Depth-First Graph Search است. برای هر وضعیت، جانشین‌ها با `successors()` تولید و به ترتیب تصادفی در پشته قرار می‌گیرند. وضعیت‌های موجود در `visited` حذف می‌شوند و مسیر جواب با والدها بازسازی می‌شود. در نتیجه یک وضعیت شامل اشغال خانه‌ها به اضافه ماسک دریچه‌های باز دوباره کشف نمی‌شود. این روش تصادفی‌بودن انتخاب مسیر را دارد اما بهینگی هزینه یا کوتاه‌ترین مسیر را تضمین نمی‌کند.

۳۰۴ دیگر الگوریتم‌ها

DFS

جست‌وجوی عمقی با پشته؛ یک جواب پیدا می‌کند ولی بهینگی هزینه را تضمین نمی‌کند.

BFS

جست‌وجوی سطحی؛ در صورت اجرای کامل، مسیری با کمترین تعداد اعمال پیدا می‌کند، نه لزوماً کمترین هزینه وزن‌دار.

IDS

جست‌وجوی عمقی با افزایش تدریجی حد عمق؛ چند بار حالت‌های کم‌عمق را بررسی می‌کند و ممکن است زمان‌بر شود.

UCS

وضعیت با کمترین هزینه طی‌شده را گسترش می‌دهد؛ با هزینه‌های نامنفی، معیار مرجع برای هزینه بهینه است.

A*

از $f(s) = g(s) + h(s)$ استفاده می‌کند؛ با تابع تخمین مجاز و پیاده‌سازی صحیح، جواب کم‌هزینه را حفظ می‌کند.

RBFS

جست‌وجوی نخست‌بهترین بازگشتی با محدودیت حافظه و امکان بازدید دوباره شاخه‌ها؛ در آزمایش محدود ممکن است به Timeout یا سقف گسترش برسد.

فصل ۵

بازاجرای مثال استاد

۱۰۵ شبکه اولیه و مسیر Greedy

شبکه مورد استفاده:

$$S_{\cdot} = \begin{bmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۰ & ۱ \\ ۱ & ۰ \end{bmatrix}.$$

تمام دریچه‌های آن در ابتدا بسته‌اند. ترتیب گام‌ها و هزینه‌های Greedy مطابق خروجی مدل تثبیت‌شده به صورت زیر است:

جدول ۱۰۵: ردیابی هفت عمل Greedy استاد

گام	دریچه / رویداد	هزینه عمل	هزینه تجمعی
۱	ستون ۱، ردیف ۱ به ۲	۵	۵
۲	ستون ۱، ردیف ۲ به ۳	۵	۱۰
۳	ستون ۱، ردیف ۳ به ۴؛ مقصد پر و دریچه منتظر	۹	۱۹
۴	خروجی ستون ۱؛ تخلیه پیوسته ستون	۵	۲۴
۵	ستون ۲، ردیف ۲ به ۳؛ مقصد پر و دریچه منتظر	۹	۳۳
۶	ستون ۲، ردیف ۳ به ۴؛ ادامه جریان از بالا	۹	۴۲
۷	خروجی ستون ۲؛ تخلیه پیوسته باقی‌مانده	۵	۴۷

$$C_{\text{Greedy}} = ۵ + ۵ + ۹ + ۵ + ۹ + ۹ + ۵ = \boxed{۴۷}.$$

در این مسیر هیچ عمل افقی انجام نشده است. دریچه مسدود در گام‌های سوم و پنجم بسته نمی‌شود، بلکه در انتظار ظرفیت می‌ماند. پرداخت هزینه خروجی در گام چهارم و هفتم به یک جریان پیوسته تعلق دارد و برای تک‌تک خانه‌هایی که در همان عمل خالی شده‌اند تکرار نمی‌شود.

۲.۵ مقایسه با جواب کم‌هزینه

برای مثال استاد، UCS و A* هر دو هزینه ۳۴ و ۱۰ عمل تولید کرده‌اند؛ درحالی‌که Greedy هفت عمل و هزینه ۴۷ دارد. بنابراین کم بودن تعداد اعمال به معنای کم بودن هزینه وزن‌دار نیست. نسبت هزینه Greedy به مقدار بهینه برابر است با:

$$\frac{۴۷}{۳۴} \simeq ۱٫۳۸۲۴.$$

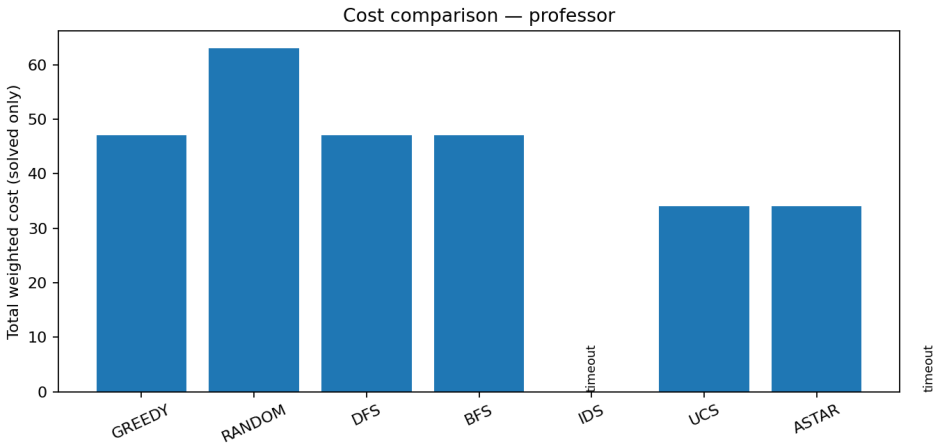
این مقایسه به تنهایی نشان می‌دهد روش تخلیه مستقل ستون‌ها همیشه بهترین انتخاب از نظر هزینه نیست.

۳.۵ جدول هر هشت الگوریتم در اجرای یکپارچه

جدول زیر **مربوط به اجرای** `run_project.py` **با سقف ۱۰ ثانیه و پایش حافظه** است. نتیجه اجرای مستقیم خط فرمان با پایش متفاوت و Seed متفاوت نباید با همین جدول ادغام شود.

جدول ۲.۵: مثال استاد در اجرای یکپارچه هفته دوم

روش	وضعیت	هزینه	اعمال	گسترش
Greedy	موفق	۴۷	۷	۷
Random	موفق	۶۳	۲۱	۲۱
DFS	موفق	۴۷	۷	۷
BFS	موفق	۴۷	۷	۲۴۷۳
IDS	زمان تمام شد	–	–	۱۸۷۷۰
UCS	موفق	۳۴	۱۰	۵۶۶۱
A* Basic	موفق	۳۴	۱۰	۵۳۲۶
RBFS	زمان تمام شد	–	–	۱۲۱۷۲



شکل ۱۰۵: مقایسه هزینه روش‌های پاسخ‌یافته در اجرای یکپارچه مثال استاد؛ خط‌چین، هزینه بهینه UCS را نشان می‌دهد.

۴۰۵ چرا نتیجه Random در دو دستور متفاوت بود؟

در اجرای مستقیم با `--random-seed 42`، هزینه Random برابر ۴۵ و تعداد اعمال ۱۱ بود. در اجرای یکپارچه، از Seed مشتق‌شده از مقدار پایه و کد وضعیت اولیه استفاده شد؛ برای مثال استاد مقدار مؤثر ۲۹۱۹۴۴۱۸۶ بود و نتیجه ۲۱ عمل با هزینه ۶۳ شد. هر دو مسیر از نظر قوانین حرکت معتبرند، اما برای مقایسه و بازتولید باید Seed مؤثر دقیق ثبت شود.

فصل ۶

طراحی تابع تخمین هزینه به روش Relaxation

۱۰۶ تابع Basic و ایده پیشنهادشده در کلاس

تابع پایه فقط هزینه اجتناب‌ناپذیر خروج از شبکه را برآورد می‌کند:

$$h_{\text{basic}}(s) = \begin{cases} ۰ & s \text{ هدف باشد}, \\ ۳ & \text{فقط نیم‌واحد شن باقی باشد}, \\ ۵ & \text{در غیر این صورت}. \end{cases}$$

در کلاس، تخمین ساده دیگری با شمردن خانه‌ها پیشنهاد شد:

$$h_{\text{count}}(s) = ۵N_{\text{full}}(s) + ۳N_{\text{half}}(s).$$

ایده این فرمول واضح است، اما در مدل جریان پیوسته برای همه وضعیت‌ها مجاز نیست. برای مثال اگر در یک ستون دو خانه پر داشته باشیم و دریچه بین آن‌ها از قبل باز باشد، خروج پیوسته ممکن است با تنها یک عمل خروج با هزینه ۵ انجام شود؛ درحالی‌که تابع شمارشی مقدار ۱۰ می‌دهد. بنابراین امکان $h(s) > h^*(s)$ وجود دارد. بسته بودن همه دریچه‌ها در ابتدا این مشکل را برای وضعیت‌های میانی A* برطرف نمی‌کند.

۲۰۶ فرمول پیشنهادی و پیاده‌سازی آن

برای هر مرز بین ردیف‌های r و $r+۱$، مقدار شن در ردیف‌های بالاتر و وضعیت دریچه‌های عمودی روی آن مرز بررسی می‌شود. مقدار $B_r(s)$ را برحسب نیم‌واحدهای شن بالای مرز تعریف می‌کنیم. اگر مرز دریچه عمودی بازی داشته باشد، هزینه تخمینی آن صفر است؛ در غیر این صورت:

$$L_r(s) = \begin{cases} ۰ & B_r = ۰ \text{ یا دریچه عمودی باز باشد}, \\ ۳ & B_r = ۱ \text{ و همه دریچه‌های آن مرز بسته باشند}, \\ ۵ & B_r \geq ۲ \text{ و همه دریچه‌های آن مرز بسته باشند}. \end{cases}$$

در نهایت:

$$\boxed{h_{\text{relax}}(s) = E(s) + \sum_{r=۱}^{R-۱} L_r(s)}, \qquad E(s) = \begin{cases} ۰ & \text{هدف}, \\ ۳ & \text{فقط نیم‌واحد شن}, \\ ۵ & \text{مقدار بیشتر}. \end{cases}$$

این تابع مقدار شن را واقعاً حرکت نمی‌دهد؛ فقط حداقل هزینه‌های اجتناب‌ناپذیر تا خالی شدن کل شبکه را برآورد می‌کند. در مثال اولیه استاد، سه مرز بسته با حداقل سهم ۵ و هزینه خروج حداقلی ۵ داریم:

$$h_{\text{relax}}(S.) = ۵ + ۵ + ۵ + ۵ = ۲۰, \quad h_{\text{basic}}(S.) = ۵, \quad h^*(S.) = ۳۴.$$

۳.۶ استدلال برای Admissibility

برای تخلیه تمام شن‌های بالای هر مرز، آن شن باید دست‌کم یک بار از مرز عبور کند. اگر هیچ دریچه عمودی روی مرز باز نباشد، دست‌کم یک عمل باز کردن روی آن مرز لازم است. برای نیم‌واحد، کمترین هزینه ممکن ۳ است. برای یک واحد یا بیشتر، حداقل یک باز کردن پرهزینه‌تر یا چند باز کردن لازم است، و کران پایین ۵ به دست می‌آید. در حالتی که دریچه‌ای قبلاً باز باشد، سهم صفر انتخاب می‌شود تا هزینه‌ای دوباره دریافت نشود.

از سوی دیگر، عمل‌های باز کردن دریچه‌های مرزهای متفاوت و عمل خروج، اعمال متمایزی هستند؛ در نتیجه کران‌های اجتناب‌ناپذیر را می‌توان با هم جمع کرد. هزینه عمل‌های افقی و باز کردن‌های اضافی در تخمین نادیده گرفته می‌شود. بر مبنای قواعد همین نسخه مدل، این استدلال کران پایین هزینه باقی‌مانده را توضیح می‌دهد:

$$h_{\text{relax}}(s) \le h^*(s).$$

این ادعا برای قواعد هزینه و جریان پیوسته تعریف‌شده در این مدل مطرح است؛ اگر قواعد تغییر کنند باید دوباره بررسی شود.

۴.۶ اعتبارسنجی محاسباتی و اثر بر A*

تست‌ها بر روی همه ۲۴۲٬۱۱ وضعیت پایدار و ۶۹۴٬۷۳ انتقال شبکه ۲ × ۳ انجام شدند و نقض Admissibility یا Consistency گزارش نشد. آزمون جامع برای شبکه کوچک، پشتیبان استدلال است اما به تنهایی اثباتی مستقل برای تمام ابعاد نیست.

در مثال استاد، A* Basic با هزینه ۳۴ تعداد ۵۳۲۶ وضعیت گسترش داد، درحالی‌که A* Relaxation با همان هزینه ۳۴ فقط ۲۹۶۳ وضعیت گسترش داد؛ کاهش تعداد گسترش‌ها تقریباً ۴۴٫۴ درصد است. این نتیجه صرفاً کیفیت عملکرد روی همان مثال را بیان می‌کند، نه بهبود تضمین‌شده برای همه ورودی‌ها.

فصل ۷

یادگیری ماشین برای تخمین هزینه باقیمانده

۱۰۷ تولید داده برچسب‌دار

برای هر وضعیت s، برچسب آموزش $h^*(s)$ است؛ یعنی کمترین هزینه وزن‌دار برای رسیدن از آن وضعیت به شبکه خالی. این مقادیر با یک اجرای Reverse Dijkstra بر گراف حالت‌های پایدار شبکه ۲ × ۳ محاسبه شدند. تعداد آرایش‌های اولیه برابر $۷۲۹ = ۳^۶$ و تعداد وضعیت‌های مدل، با در نظر گرفتن دریچه‌های باز، ۲۴۲٬۱۱ است.

ویژگی‌های هر نمونه شامل مقدار داخلی شن هر خانه، بیت‌های باز بودن دریچه‌ها، مجموع شن ردیف‌ها و مجموع شن کل شبکه است. مدل RandomForestRegressor با ۳۰ درخت آموزش داده شد. ویژگی‌های وابسته به اندازه شبکه باعث می‌شوند مدل آموزش‌دیده ۲ × ۳ را نتوان مستقیماً به شبکه ۲ × ۴ یا ۲ × ۵ تعمیم داد.

۲۰۷ تقسیم داده و جلوگیری از هم‌پوشانی آرایش‌ها

تقسیم آموزش/آزمون با GroupShuffleSplit و بر مبنای آرایش مقدار شن انجام شد. تمام وضعیت‌هایی که اشغال یکسان ولی ماسک دریچه‌های متفاوت دارند، در یک گروه قرار گرفتند. بدین ترتیب یک آرایش شن ثابت هم‌زمان در آموزش و آزمون حضور ندارد. در سه تقسیم مستقل، هر بار ۵۸۳ گروه اشغال برای آموزش و ۱۴۶ گروه برای آزمون استفاده شدند.

البته در آزمایش جست‌وجو، وضعیت اولیه از گروه‌های آزمون انتخاب می‌شود، ولی بعضی وضعیت‌های میانیِ مسیر ممکن است در آرایش‌های آموزش دیده شده باشند. بنابراین ادعای «کاملاً ندیده بودن تک‌تک وضعیت‌های طی جست‌وجو» مطرح نیست.

۳.۷ معیارهای پیش‌بینی

جدول ۱.۷: ارزیابی سه تقسیم آموزش/آزمون مدل جنگل تصادفی

Seed	آموزش	آزمون	MAE	RMSE	R^2
۴۲	۸۹۳۹	۲۳۰۳	۹۷۳٫۰	۲۳۶٫۱	۹۵۸٫۰
۴۳	۸۹۰۷	۲۳۳۵	۹۲۶٫۰	۱۷۲٫۱	۹۶۳٫۰
۴۴	۸۰۹۰	۳۱۵۲	۱۷۰٫۱	۵۷۲٫۱	۹۳۹٫۰

در همین ترتیب ،Seed نسبت پیش‌بینی‌های بیشتر از هزینه بهینه حدود ۳۷٫۱۳، ۵۶٫۴۹ و ۴۲٫۷۰ درصد بود. بنابراین R^2 بالا، Admissible بودن پیش‌بینی را تضمین نمی‌کند.

۴.۷ دو روش استفاده از ML در A*

ML Direct
پیش‌بینی رگرسیون مستقیماً جای h در $f = g + h$ قرار می‌گیرد. ممکن است جست‌وجو را بسیار هدایت کند، ولی چون بیش‌برآورد وجود دارد، تضمین بهینگی از بین می‌رود.

ML Safe Tie
کلید اصلی صف همان $g + h_{\text{relax}}$ است. مقدار ML فقط میان وضعیت‌هایی که کلید اصلی برابر دارند، ترتیب را تعیین می‌کند. به این ترتیب در شرایط صحت A* و Relaxation، تضمین بهینگی کلید اصلی حفظ می‌شود.

پیش‌بینی دسته‌ای و Cache محلی هر جست‌وجو برای کاهش هزینه فراخوانی مدل استفاده شده‌اند. زمان آموزش و بارگذاری مدل از زمان جست‌وجو جداست؛ اما زمان پیش‌بینی در خود جست‌وجو محاسبه می‌شود.

فصل ۸

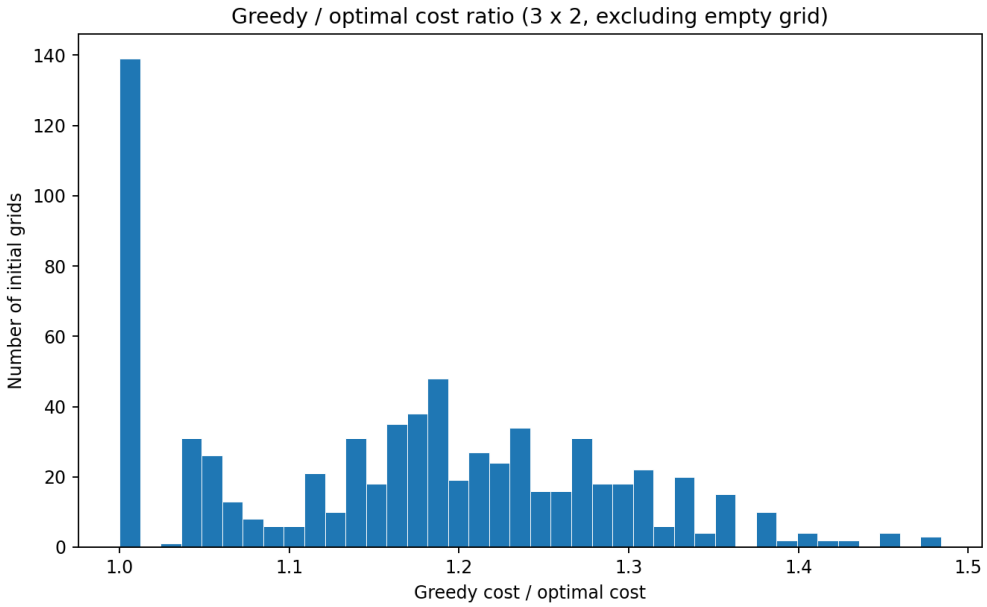
نتایج جامع، نمودارها و مقایسه روش‌ها

۱۰۸ تحلیل دقیق تمام شبکه‌های اولیه ۳×۲

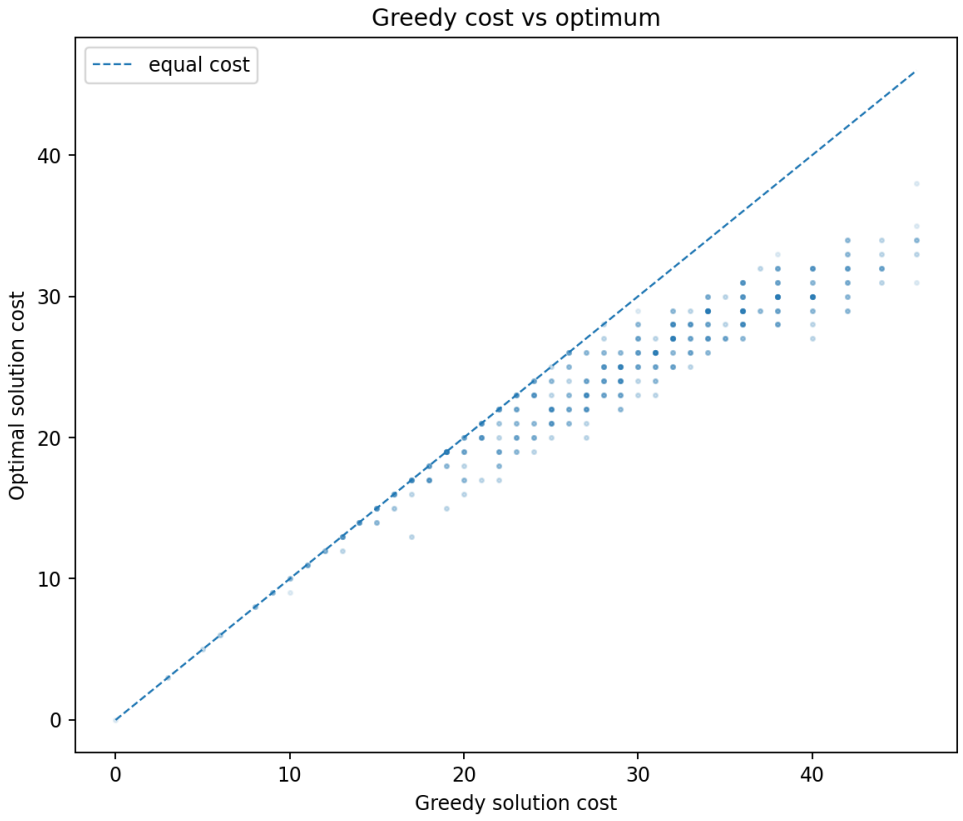
شبکه ۲ × ۳ دارای ۷۲۹ آرایش اولیه است که در آن همه دریچه‌ها بسته‌اند. گراف حالتِ مدل اصلاح‌شده ۲۴۲٬۱۱ وضعیت پایدار و ۶۹۴٬۷۳ انتقال وزن‌دار داشت. محاسبه Reverse Dijkstra هزینه بهینه تمام آرایش‌های اولیه را به دست داد.

جدول ۱۰۸: خلاصه تحلیل جامع ۳×۲ بر اساس مدل جریان پیوسته

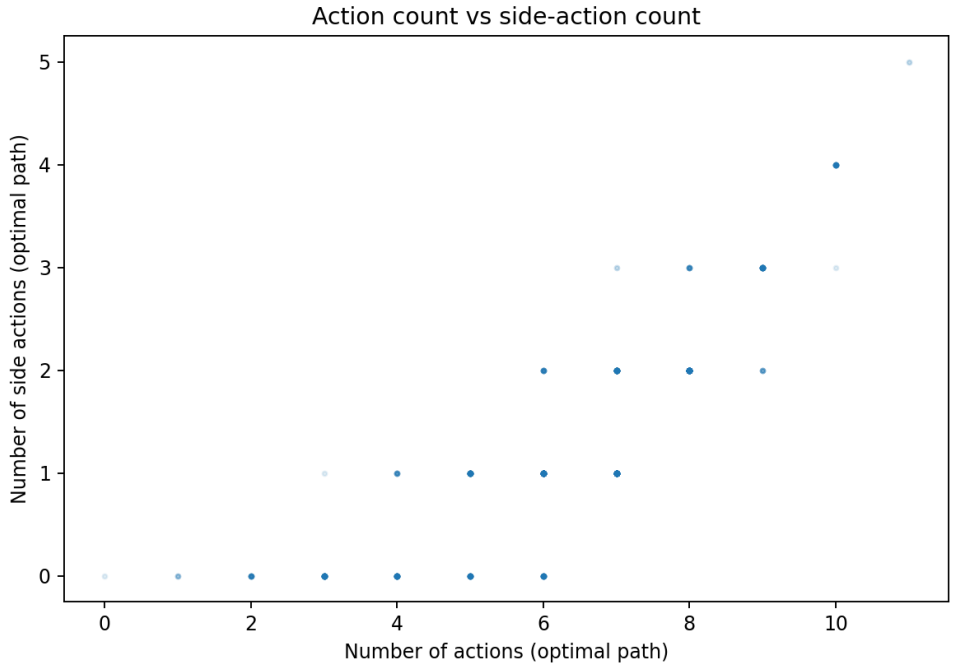
معیار	مقدار
تعداد شبکه‌های اولیه	۷۲۹
تعداد وضعیت‌های پایدار	۲۴۲٬۱۱
تعداد انتقال‌ها	۶۹۴٬۷۳
میانگین هزینه Greedy	۲۹۶۳٫۲۸
میانگین هزینه بهینه	۹۰۱۲٫۲۳
میانگین نسبت Greedy به بهینه (بدون هدف خالی)	۱۶۵۷۴۱٫۱
میانه نسبت	۱۷۳۹۱۳٫۱
بیشترین نسبت مشاهده‌شده	۴۸۳۸۷۱٫۱
سهم آرایش‌هایی که Greedy بهینه بود	۲۰۴۴٫۱۹ درصد



شکل ۱۰۸: هیستوگرام نسبت هزینه Greedy به هزینه بهینه برای ۷۲۸ آرایش غیرخالی شبکه ۳×۲.



شکل ۲۰۸: نمودار پراکندگی هزینه Greedy و هزینه بهینه برای آرایش‌های اولیه ۳×۲.



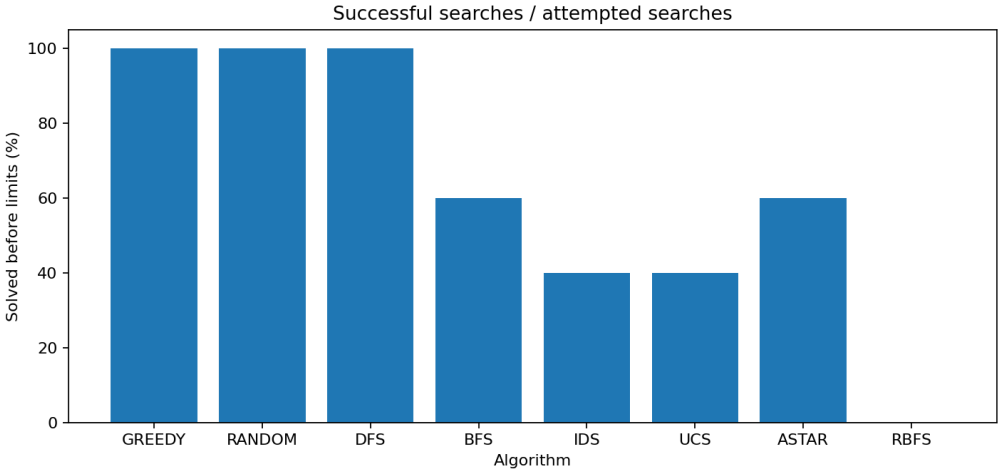
شکل ۳۰۸: تعداد کل اعمال و تعداد اعمال افقی در یک مسیر بهینه انتخاب‌شده.

۲۰۸ مقایسه هشت الگوریتم روی پنج ورودی نمونه ۵×۲

برای پنج ورودی مشترک ۲ × ۵، محدودیت زمانی ۱۰ ثانیه و سقف ۱۰۰٬۰۰۰ گسترش منظور شد. تعداد اجراهای پاسخ‌یافته به ترتیب چنین بود:

جدول ۲۰۸: تعداد پاسخ‌های ثبت‌شده در بنچمارک پنج ورودی ۵×۲

روش	پاسخ‌یافته / کل	وضعیت قابل تفسیر
Greedy	۵ / ۵	روش مبنا
Random	۵ / ۵	وابسته به Seed
DFS	۵ / ۵	بدون تضمین هزینه بهینه
BFS	۵ / ۳	دو اجرا محدود شده
IDS	۵ / ۲	سه اجرا محدود شده
UCS	۵ / ۲	سه اجرا محدود شده
A* Basic	۵ / ۳	دو اجرا محدود شده
RBFS	۵ / ۰	پنج اجرا محدود شده



شکل ۴.۸: نرخ یافتن پاسخ در نمونه پنج‌تایی ۵×۲؛ این نمونه برای نتیجه‌گیری کلی کافی نیست.

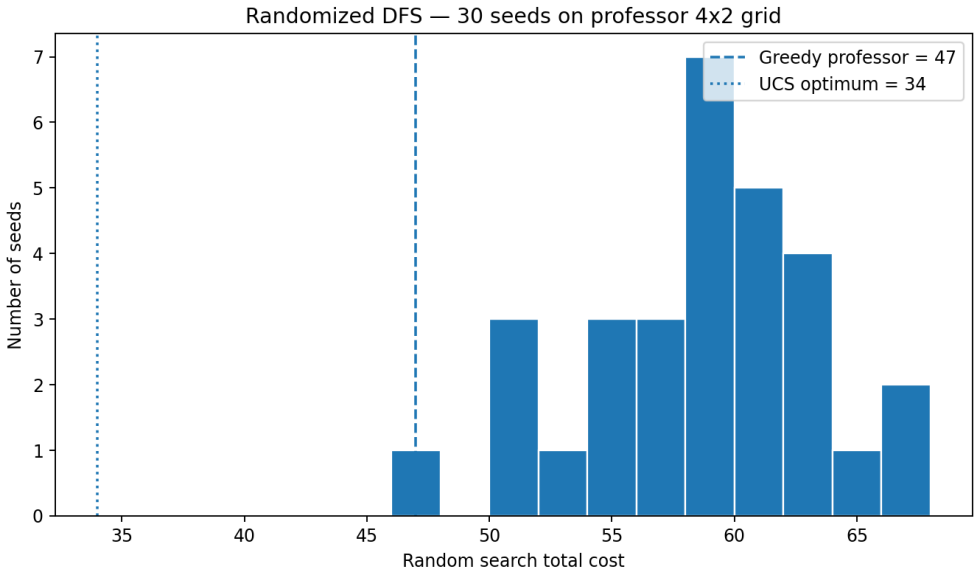
تنها دو ورودی از این پنج مورد دارای پاسخ تکمیل‌شده مشترک از Greedy و UCS بودند؛ بنابراین نمودار نسبت هزینه این نمونه نباید به عنوان تحلیل جامع تمام ۵۹۰۴۹ = ۳۱۰ آرایش اولیه گزارش شود. به دلیل بزرگ شدن فضای حالت با دریچه‌های ماندگار، تحلیل کامل مدل جدید برای ۲ × ۵ در این پروژه انجام نشد. آمار تحلیل جامع ۲ × ۵ گزارش پیشین متعلق به مدل قدیمی است.

۳.۸ آزمایش ۳۰ Seed برای Random روی مثال استاد

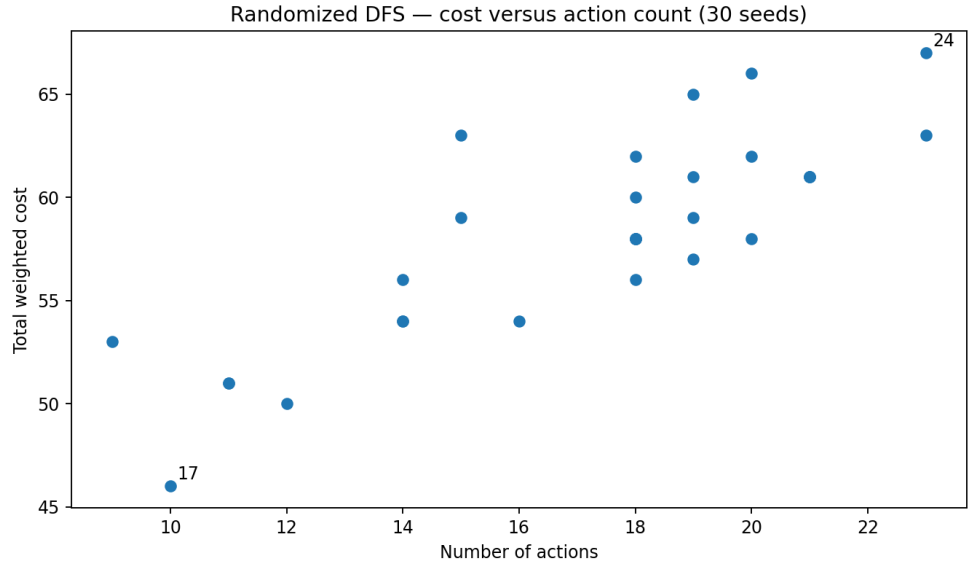
برای ارزیابی حساسیت به انتخاب تصادفی، ۳۰ اجرای مستقل با Seedهای ۱ تا ۳۰ از یک شبکه ثابت استاد ذخیره شد. آمار محاسبه‌شده از فایل‌های JSON همان اجراها:

جدول ۳.۸: آمار توزیع هزینه Random روی ۳۰ Seed

معیار	مقدار
تعداد اجراهای ثبت‌شده	۳۰
میانگین هزینه	۰۶۷٫۵۸
میانه هزینه	۵۸
کمترین هزینه	۴۶ (Seed= ۱۷)
بیشترین هزینه	۶۷ (Seed= ۲۴)
انحراف معیار نمونه هزینه	۹۳۴٫۴
میانگین تعداد اعمال	۰۶۷٫۱۷
کمترین و بیشترین تعداد اعمال	۹ و ۲۳



شکل ۵.۸: توزیع هزینه Random روی ۳۰ Seed؛ خط‌های مرجع، هزینه Greedy=۴۷ و بهینه UCS=۳۴ هستند.



شکل ۶.۸: رابطه تعداد اعمال و هزینه؛ Random تعداد اعمال کمتر الزاماً هزینه کمتر نمی‌دهد.

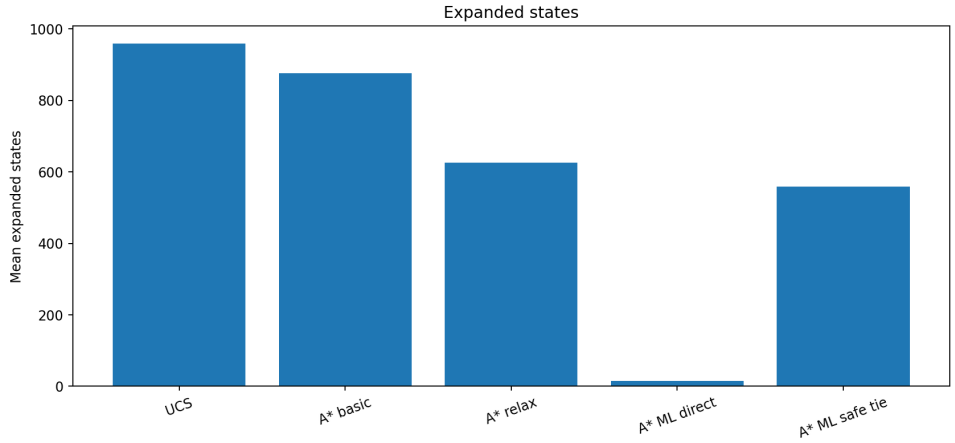
کمترین هزینه در این ۳۰ اجرا، ۴۶ است؛ این عدد از هزینه Greedy استاد (۴۷) کمتر ولی از مقدار بهینه ۳۴ بیشتر است. این مجموعه شامل Seed=۴۲ نیست؛ نتیجه اجرای مستقیم Seed=۴۲ که هزینه ۴۵ بود، آزمایشی جداگانه است و نباید جزو آمار همین ۳۰ اجرا شمرده شود.

۴.۸ ارزیابی پنج روش Heuristic روی ۶۰ ورودی

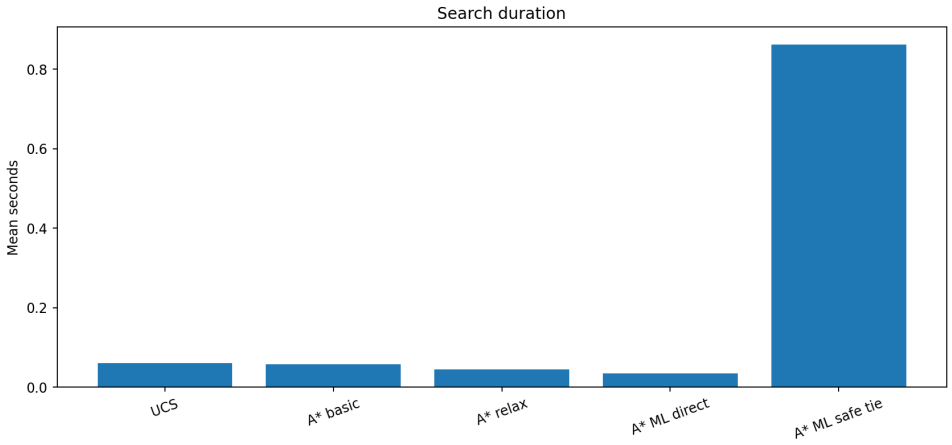
سه تقسیم داده با Seedهای ۴۲، ۴۳ و ۴۴ و برای هرکدام ۲۰ وضعیت اولیه آزمون استفاده شد. جدول نخست از اجرای بدون tracemalloc است و برای مقایسه زمان مناسب‌تر است:

جدول ۴.۸: عملکرد پنج روش روی ۶۰ آزمایش جست‌وجو، بدون پایش حافظه

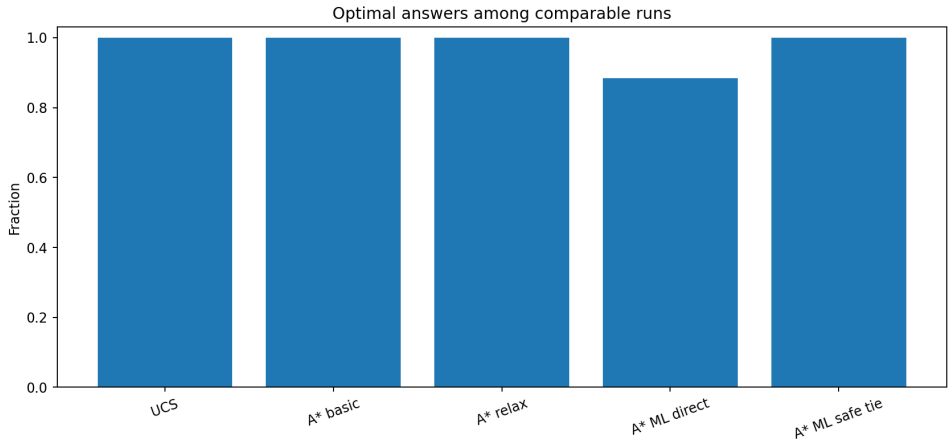
روش	میانگین گسترش	میانگین زمان (ثانیه)	جواب بهینه
UCS	۷۸٫۹۵۹	۰۶۰۲٫۰	۶۰/۶۰
A* Basic	۸۸٫۸۷۶	۰۵۷۵٫۰	۶۰/۶۰
A* Relax	۵۳٫۶۲۶	۰۴۵۱٫۰	۶۰/۶۰
A* ML Direct	۷۸٫۱۴	۰۳۴۵٫۰	۶۰/۵۳
A* ML Safe Tie	۱۵٫۵۵۹	۸۶۲۸٫۰	۶۰/۶۰



شکل ۷.۸: میانگین تعداد وضعیت‌های گسترش‌یافته در ۶۰ آزمایش.



شکل ۸.۸: میانگین زمان جست‌وجو در اجرای بدون اندازه‌گیری حافظه.



شکل ۹.۸: نسبت جواب‌های بهینه، روش Direct ML در ۷ مورد هزینه بیشتر از UCS داشت.

در این مجموعه، Relaxation نسبت به Basic حدود ۲۸٫۵ درصد حالت کمتر گسترش داد و همه جواب‌ها بهینه بودند. ML Direct در ۷ آزمایش جواب زیربهینه داشت؛ در مقابل، ML Safe Tie بهینگی را حفظ کرد اما به دلیل هزینه استنتاج ML کندتر بود.

۵.۸ برآورد ضریب انشعاب مؤثر b^*

برای سنجش کارایی جست‌وجو، ضریب انشعاب مؤثر b^* را محاسبه می‌کنیم. این شاخص معادل ضریب انشعاب یک درخت یکنواخت با عمق جواب d است که تقریباً همان تعداد گره بررسی‌شده در اجرای واقعی را داشته باشد. مقدار آن از حل عددی رابطه زیر به دست می‌آید:

$$N_{\text{eff}} = ۱ + b^* + (b^*)^۲ + \cdots + (b^*)^d = \sum_{k=۰}^{d} (b^*)^k.$$

در کد حاضر، شمارنده expanded گره هدف را در بر نمی‌گیرد؛ بنابراین برای برآورد مبتنی بر گسترش، مقدار $N_{\text{eff}} = \text{expanded} + ۱$ قرار داده‌ایم. برای مثال، اجرای A* Basic روی مسئله استاد $d = ۱۰$ و expanded $= ۵۳۲۶$ دارد:

$$۱ + b^* + \cdots + (b^*)^{۱۰} = ۵۳۲۷ \quad \Longrightarrow \quad b^* \simeq ۲٫۲۲۱۶۸۶.$$

این معادله با روش Bisection حل شده است. مقدار generated نیز در اجرای مثال استاد موجود است؛ اما در پیاده‌سازی ما این ستون تعداد انتقال‌های جانشین تولیدشده، شامل تکراری‌ها است، نه لزوماً تعداد گره‌های یکتای ذخیره‌شده در مرز جست‌وجو. ازاین‌رو مقدار مبتنی بر آن را صرفاً برای شفافیت می‌آوریم و نباید بدون توضیح با تعریف کتابی ضریب انشعاب مؤثر برابر بدانیم.

جدول ۵.۸: ضریب انشعاب مؤثر در مثال استاد، فقط الگوریتم‌هایی که جواب یافته‌اند

روش	عمق d	Expanded	b^*_{expanded}	$b^*_{\text{generated}}$
Greedy	۷	۷	۱٫۰۰۰	۱٫۰۰۰
Random	۲۱	۲۱	۱٫۰۰۰	۱٫۱۲۷
DFS	۷	۷	۱٫۰۰۰	۱٫۸۷۴
BFS	۷	۲۴۷۳	۲٫۲۷۲	۳٫۸۷۷
UCS	۱۰	۵۶۶۱	۲٫۳۶۲	۴٫۶۹۲
A* Basic	۱۰	۵۳۲۶	۲٫۲۲۲	۲٫۸۲۶
A* Relaxation	۱۰	۲۹۶۳	۲٫۰۸۴	—

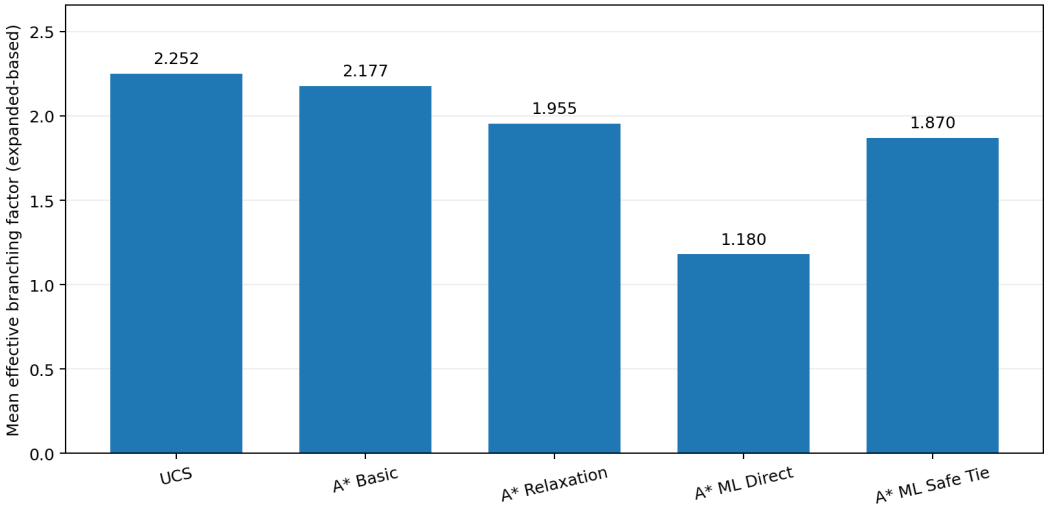
ردیف A* Relaxation بر پایه اجرای جداگانه همان مثال با تابع واهلش است؛ شمارنده generated آن در خروجی مورد استفاده موجود نبود، بنابراین برای آن عددی فرض نشده است. مقدار b^* در این ردیف از معادله $۱ + b + \cdots + b^{۱۰} = ۲۹۶۴$ به دست آمده است.

در روش‌های Greedy، Random و DFS مقدار $b^*_{\text{expanded}} = ۱$ حاصل از تقریباً برابر بودن تعداد گسترش‌ها با طول مسیر جواب است. این مقدار **به معنای بهینه بودن مسیر یا برتری این روش‌ها نیست**؛ به‌خصوص هزینه Random به انتخاب تصادفی مسیر وابسته است. برای اجراهای بدون جواب یا دارای Timeout نیز d مشخص نیست و ضریب انشعاب مؤثر محاسبه نشده است.

برای سنجش اثر تابع آرویی، ضریب b^*_{expanded} را برای هر یک از ۶۰ مسئله آزمون شبکه ۲×۳ جداگانه محاسبه کرده و سپس میانگین و میانه گرفته‌ایم. پس اعداد جدول زیر میانگین ضرایب هر اجرا هستند، نه نتیجه جایگذاری میانگین Expanded در یک معادله واحد.

جدول ۶.۸: ضریب انشعاب مؤثر بر اساس گسترش گره‌ها در ۶۰ آزمایش

روش	تعداد حل‌شده	میانگین b^*	میانه b^*
UCS	۶۰	۲۵۲٫۲	۱۳۵٫۲
A* Basic	۶۰	۱۷۷٫۲	۰۸۹٫۲
A* Relaxation	۶۰	۹۵۵٫۱	۹۰۵٫۱
A* ML Direct	۶۰	۱۸۰٫۱	۱۴۵٫۱
A* ML Safe Tie	۶۰	۸۷۰٫۱	۸۱۲٫۱



شکل ۱۰.۸: میانگین ضریب انشعاب مؤثر مبتنی بر گره‌های گسترش‌یافته در ۶۰ مسئله آزمون.

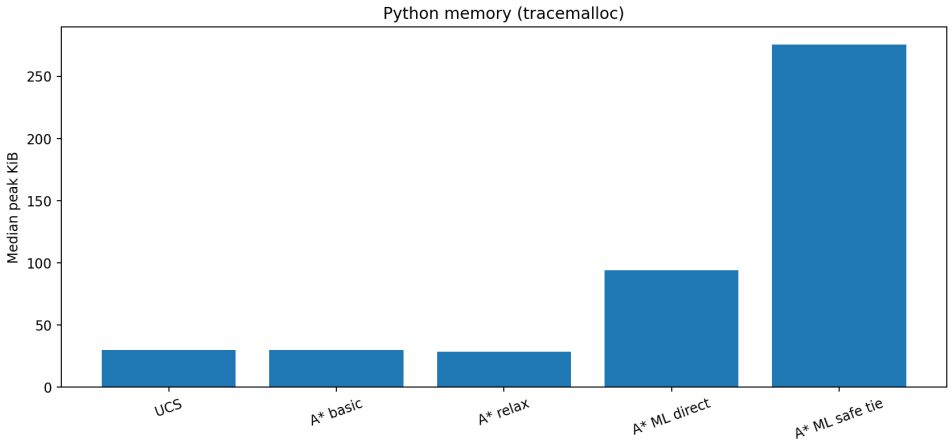
نتایج نشان می‌دهد Relaxation نسبت به Basic ضریب انشعاب مؤثر کمتری داشته است (حدود ۱٫۹۵۵ در برابر ۲٫۱۷۷) و هم‌زمان هزینه بهینه را در همه ۶۰ مسئله حفظ کرده است. مقدار پایین‌تر ML Direct به‌تنهایی نشانه بهتر بودن آن نیست، زیرا این روش در ۷ مورد جواب زیربهینه تولید کرده است. همچنین این ضریب به عمق جواب وابسته است؛ از این رو مقایسه روش‌هایی با طول مسیر متفاوت باید همراه با هزینه جواب و تعداد گسترش‌ها انجام شود.

۶.۸ آزمایش حافظه با سقف ۶۰ ثانیه

اندازه‌گیری دوم با `--measure-memory` و سقف ۶۰ ثانیه انجام شد تا مسئله‌های کند هم تکمیل شوند. هر پنج روش در ۶۰ اجرا جواب پیدا کردند، اما ML Direct فقط در ۵۳ اجرا به هزینه بهینه رسید. جدول زیر میانه تخصیص ثبت‌شده توسط *Python* را گزارش می‌کند؛ این مقدار مصرف کل RAM فرایند نیست:

جدول ۷.۸: آزمایش مستقل حافظه؛ زمان‌ها تحت تأثیر tracemalloc هستند

روش	میانه حافظه (KiB)	میانگین زمان (ثانیه)	بهینه / حل‌شده
UCS	۰۰٫۳۰	۸۰۳٫۰	۶۰/۶۰
A* Basic	۰۰٫۳۰	۷۴۸٫۰	۶۰/۶۰
A* Relax	۵۹٫۲۸	۵۸۱٫۰	۶۰/۶۰
A* ML Direct	۹۶٫۹۳	۲۶۷٫۰	۶۰/۵۳
A* ML Safe Tie	۸۱٫۲۷۵	۱۸۹٫۷	۶۰/۶۰



شکل ۱۱.۸: میانه حافظه تخصیص‌یافته Python در اجرای مستقل با tracemalloc.

نتیجه اصلی این است که A* Relaxation در آزمایش‌های حاضر تعادل مناسبی میان حفظ بهینگی، تعداد گسترش‌ها، زمان و حافظه نشان داد. برتری آن به معنای بهترین بودن مطلق در همه شبکه‌ها نیست.

فصل ۹

اعتبارسنجی، محدودیت‌ها و بازتولید نتایج

۱۰۹ آزمون‌های صحت پس از تغییرات

برای مدل جریان پیوسته، آزمون‌هایی درباره بازماندن دریچه مسدود، انتقال خودکار، تخلیه زنجیره‌ای از خروجی، حرکت افقی نیم‌واحدی، کدگذاری ماسک دریچه، قرارداد = ۱پر و بازتولید مثال استاد نوشته شد. تست‌های مستقل Relaxation روی شبکه‌های کوچک و تست‌های Random درباره نبود حالت تکراری در مسیر جواب و بازتولیدپذیری Seed نیز افزوده شد. در آخرین اجرای یکپارچه همراه ،Random **۴۰ تست پاس شدند.** این عدد شمار آزمون‌های اجراشده است و ممکن است برخی آزمون‌ها پوشش مشابهی داشته باشند.

۲۰۹ محدودیت‌های تجربی

۱. مدل پایه بدون آب و بتن است؛ هیچ نتیجه‌ای درباره افزونه‌های پیشنهادی آب و بتن ارائه نمی‌شود.

۲. نتایج جامع فقط به همه ورودی‌های اولیه ۲ × ۳ تعلق دارند؛ برای ۲ × ۵ تنها بنچمارک نمونه‌ای داریم.

۳. مدل ML به اندازه شبکه و نسخه قوانین وابسته است. آزمایش روی ۲ × ۳ مستقیماً به شبکه بزرگ‌تر تعمیم داده نمی‌شود.

۴. Timeout یا سقف گسترش به معنی نداشتن مسیر حل نیست. برای آزمایش‌های تکمیل‌نشده هزینه‌ای جعل یا صفر فرض نشده است.

۵. اعداد tracemalloc به حافظه Python محدودند و برخی تخصیص‌های کتابخانه‌های بومی را پوشش نمی‌دهند. زمان اجرا با فعال شدن پایش حافظه تغییر می‌کند.

۶. در محاسبه ضریب انشعاب مؤثر برای ۶۰ آزمایش، از تعداد گره‌های گسترش‌یافته به اضافه گره هدف استفاده شده است. داده تعداد جانشین‌های تولیدشده برای این مجموعه ثبت نشده و نباید تخمین مبتنی بر گسترش را بی‌قیدوشرط معادل تعریف کتابیِ تعداد گره‌های تولیدشده دانست.

۰۷. مسیرهای Random به Seed وابسته‌اند؛ روش تصادفی در یک نمونه ممکن است از Greedy بهتر یا بدتر باشد.

۰۸. تغییر قواعد مدل از هفته اول به هفته دوم، مقایسه مستقیم آمار جامع قدیمی ۲ × ۵ با خروجی‌های فعلی را نامعتبر می‌کند.

۳.۹ فایل‌های اصلی و نقش آن‌ها

فایل	نقش در نسخه هفته دوم
sandgrid/model.py	قوانین جریان پیوسته، انتقال خودکار، وضعیت دریچه‌ها و هزینه‌ها
sandgrid/search.py	الگوریتم‌های جست‌وجو، Random Greedy، و انتخاب تابع تخمین
sandgrid/heuristics.py	Basic، Relaxation ویژگی‌های ML و تخمین یادگرفته‌شده
sandgrid/cli.py	قرارداد ورودی، گزینه‌های Random و Heuristic در فرمان‌ها
sandgrid/visualize.py	انیمیشن HTML و GIF با نمایش دریچه باز و انتقال‌های مستقیم/خودکار
sandgrid/batch.py	محاسبه بهینه‌های دقیق با Dijkstra Reverse برای شبکه‌های کوچک
sandgrid/benchmark.py	اجرای قابل تکرار بنچمارک الگوریتم‌ها و ثبت Seed
run_project.py	اجرای یکپارچه، آزمون‌ها، نمودارها و گزارش هشت الگوریتم
scripts/train_ml_heuristic.py	تولید داده و آموزش Forest Random
scripts/compare_heuristics.py	آزمایش مقایسه‌ای روش‌های UCS و A*
scripts/final_heuristic_study.py	اجرای سه Seed، ادغام نتایج و اندازه‌گیری اختیاری حافظه
scripts/calc_effective_branching.py	حل عددی ضریب انشعاب مؤثر با روش جست‌وجوی دودویی
tests/	آزمون‌های مدل، جست‌وجو، Random و اعتبار Heuristic

۴.۹ دستورهای اصلی بازتولید در PowerShell

```
python -m pytest -v tests

python main.py solve --example professor --algorithm greedy
python main.py solve --example professor `
    --algorithm astar --heuristic basic
python main.py solve --example professor `
    --algorithm astar --heuristic relax
```

```
python main.py solve --example professor `
    --algorithm random --random-seed 42

python run_project.py --analysis-rows 3 --analysis-cols 2 `
    --sample-size 5 --seconds 10 --seed 42 `
    --random-seed 42 --output results/project_with_random

python scripts/final_heuristic_study.py `
    --seeds 42,43,44 --count 20 --repeats 1 `
    --trees 30 --seconds 20 `
    --output results/final_heuristics

python scripts/final_heuristic_study.py `
    --seeds 42,43,44 --count 20 --repeats 1 `
    --trees 30 --seconds 60 --measure-memory `
    --output results/final_heuristics_memory_60s
```

در PowerShell، نویسه ` در انتهای خط نشان‌دهنده ادامه دستور در خط بعد است. پس از آن نباید فاصله اضافی قرار گیرد. Seed، محدودیت‌های زمان و گسترش و نسخه مدل باید در گزارش همراه نتایج ثبت شوند.

برای تولید دوباره جدول‌های b^* پس از اجرای آزمایش‌ها، این دو دستور را اجرا کنید:

```
python scripts/calc_effective_branching.py `
  --input results/project_with_random/professor/comparison.csv `
  --output results/bstar_professor.csv
python scripts/calc_effective_branching.py `
  --input results/final_heuristics/comparison_all_seeds.csv `
  --output results/bstar_heuristics.csv `
  --summary-output results/bstar_summary.csv
```

فصل ۱۰

نتیجه‌گیری هفته دوم

در هفته دوم، مسئله از یک مدل اولیه با ابهام در معنی مقادیر خانه‌ها و رفتار دریچه‌ها به یک مدل جریان پیوسته با وضعیت دریچه‌های باز و انتقال خودکار تبدیل شد. معیار کنترل اصلی، مثال استاد بود که پس از اصلاح واقعی قوانین، هزینه Greedy برابر ۴۷ با هفت عمل و هزینه کمینه UCS/A* برابر ۳۴ شد. الگوریتم تصادفی بدون بازدید دوباره وضعیت‌ها نیز اضافه شد و وابستگی آن به Seed روی ۳۰ اجرای مستقل بررسی گردید.

محاسبه ضریب انشعاب مؤثر نیز نشان داد که میانگین مقدار مبتنی بر گسترش از ۱۷۷٫۲ برای Basic A* به ۹۵۵٫۱ برای Relaxation A* کاهش یافته است.

در بخش Heuristic، تابع Basic با Relaxation مبتنی بر مرزهای ردیف مقایسه شد. Relaxation در مثال استاد و در آزمایش‌های شبکه ۲ × ۳ حالت‌های کمتری گسترش داد و بهینگی را حفظ کرد. بخش یادگیری ماشین نشان داد که یک مدل رگرسیون با دقت پیش‌بینی بالا می‌تواند گسترش حالت‌ها را به‌شدت کاهش دهد، اما استفاده مستقیم از آن تضمین بهینگی ندارد؛ در آزمایش ۶۰تایی، Direct ML هفت پاسخ زیربهینه ثبت کرد. روش Tie Safe بهینگی را حفظ کرد، ولی هزینه زمانی و حافظه بالاتری داشت.

بنابراین برای ادامه کار با تأکید بر هزینه بهینه و کارایی در اندازه‌های آزمایش‌شده، A* Relaxation انتخاب مناسبی است. برای پژوهش بعدی می‌توان به بهبود کران پایین، پیش‌بینی سریع‌تر ML و طراحی تحلیل مقیاس‌پذیرتر برای شبکه‌های بزرگ‌تر پرداخت. این پیشنهادها فراتر از نتایج نهایی ثبت‌شده هستند و نباید به عنوان دستاورد آزمایش‌شده گزارش شوند.

پیوست آ

منابع داده و پیوست‌های قابل بررسی

نتایج عددی این گزارش از فایل‌های خروجی ارائه‌شده استخراج شده‌اند، نه از برآوردهای دستیِ بدون داده. مهم‌ترین منابع محلی عبارت‌اند از:

- گزارش و خروجی مقایسه هشت الگوریتم: `project_with_random/` در پوشه `results`
- نتایج شمارش جامع شبکه ۳×۲: `exhaustive_3x2_revised/summary.json`
- خروجی‌های تصادفی: ۳۰ فایل JSON در `random_seeds/`
- خلاصه زمان و کیفیت Heuristicها: `final_heuristics/summary_all_seeds.csv`
- خلاصه حافظه در اجرای ۶۰ ثانیه‌ای، فایل:

`final_heuristics_memory_60s/summary_all_seeds.csv`

- خطای پیش‌بینی ML: `final_heuristics/training_metrics.csv`
- محاسبه ضریب انشعاب مؤثر: `bstar_professor.csv`، `bstar_heuristics.csv` و `bstar_summary.csv` (در بسته LaTeX و پوشه نتایج).

لینک چت‌های هوش مصنوعی در فایل `GPTConversationLink.txt` قرار دارد. فایل‌های خروجی و کدهای پروژه نیز در مخزن عمومی گیت‌هاب به نشانی زیر قابل دسترسی هستند:

`https://github.com/PartowRoshani/Sandboxes-Search-Project/tree/main`